

Document Processing Libraries (Multi-Format Extraction)

We compare libraries that extract text from diverse formats (PDF, DOCX, PPTX, XLSX, HTML, Markdown, images via OCR, EPUB, ZIP, CSV, JSON, XML, etc.) and produce clean text or Markdown. Emphasis is on **JavaScript/Node.js** solutions (but we also cover Python/Java alternatives). Each entry below lists supported input/output formats, OCR capability, integration, performance, adoption, license, pros/cons, and ideal use cases (open-source or commercial).

Node.js (JavaScript/TypeScript) Libraries

- **officeParser (Open Source)** – [GitHub](#) (MIT). A robust, TypeScript-compatible library for Office and related formats [\[72†L330-L339\]](#) .
- **Formats:** `.docx`, `.pptx`, `.xlsx`, OpenOffice/LibreOffice (`.odt`, `.odp`, `.ods`), PDF, RTF, CSV, Markdown, HTML [\[72†L330-L339\]](#) . (Uses [PDF.js](#) for PDFs.)
- **Outputs:** Text (plain), Markdown, HTML, CSV, RTF, PDF, and “RAG chunks” (splits) [\[72†L330-L339\]](#) . It builds an AST and can generate various output formats (`ast.to('md')` , etc.), or extract text directly.
- **OCR:** Yes – has integrated support via [Tesseract.js](#) for images embedded in documents. The CLI flag `--ocr` enables OCR on images and attachments [\[27†L49-L57\]](#) [\[27†L99-L107\]](#) . (Requires Tesseract training data; uses a worker pool.)
- **Integration:** Provided as both a Node API and CLI (npv). Example:
`officeParser.parseOffice('file.docx', { ocr: true })` . Well-documented with live AST visualizer [\[72†L330-L339\]](#) [\[72†L451-L459\]](#) . Ships via npm (`npm i officeparser`).
- **Performance:** Reasonably efficient; parses documents natively. PDF parsing can be heavy for very large files, and OCR adds overhead. Supports aborting/timeout for long parses.
- **Community & Maintenance:** Active project (495★, frequent commits). Maintained by a single author but widely used. Released as v4.0.8 (2 Aug 2023).
- **License:** MIT (open source).
- **Pros:** Very broad format support; structured AST enables high-fidelity output; built-in OCR for images; configurable (ignore notes, comments, etc.); both Node and browser usage.
- **Cons:** Heavyweight (large bundle due to PDF.js and OCR); some formats (ePub, ZIP, audio) not covered; learning curve for AST/config; Node >=18 only.
- **Use Cases:** Best for **Office-centric workflows** (Word/Excel/PowerPoint/PDF). Ideal when you need high-fidelity conversion or AST-level access. Great for generating Markdown or HTML from office docs (e.g. migrating docs, preparing data for QA, etc.).
- **markdown-js (Open Source)** – [GitHub](#) (MIT). A Node.js reimplement of Microsoft’s Markdown, focusing on converting *everything* to Markdown [\[74†L278-L287\]](#) .

- *Formats*: All formats supported by Microsoft MarkItDown [\[74†L278-L287\]](#) : plaintext, HTML, RSS, Wikipedia, YouTube URLs, Office docs (`.docx` , `.xlsx` , `.pptx`), images, audio, video, Jupyter/IPython notebooks, PDF, ZIP (with office/content inside), Outlook MSG, and more [\[74†L278-L287\]](#) . (This is by design to mirror MarkItDown features.)
- *Outputs*: Structured Markdown plus metadata (title, alt text, etc.). The `convert(file)` method returns an object like `{ title, textContent }` [\[74†L312-L320\]](#) , with Markdown-formatted `textContent` .
- *OCR*: Yes – uses Tesseract.js (via [node-tesseract-ocr](#)) for images (needs Tesseract installed) [\[74†L325-L333\]](#) . Can extract image text, and via the “llmCall” hook it can call an LLM (e.g. OpenAI) for image description/audio transcription (mirroring MarkItDown’s Azure/LLM features) [\[74†L333-L343\]](#) [\[74†L349-L358\]](#) .
- *Integration*: Easy npm install (`npm i markitdown-js`). Usage is straightforward:

```
import Markitdown from "markitdown-js";
const converter = new Markitdown();
const md = converter.convert("file.pdf");
```

(Returns a Markdown string plus metadata.) No external CLI needed, purely code. Some configuration for OCR and LLM calls is possible [\[74†L270-L279\]](#) [\[74†L325-L333\]](#) .

- *Performance*: Relies on internal libraries (Mammoth, PDF parsers, Tesseract, etc.). Suitable for single docs; large batches might be slower. LLM-based features (image/audio transcription) depend on your LLM setup.
- *Community & Maintenance*: Niche project (18★, few forks) updated ~2023. It matches MarkItDown’s roadmap and is maintained by a contributor. No published version (GitHub-only).
- *License*: MIT (open source) [\[74†L428-L431\]](#) .
- *Pros*: Covers the **widest range** of formats (basically everything MarkItDown does) [\[74†L278-L287\]](#) . Directly outputs Markdown. Includes image/audio “insights” if you wire an LLM. Integrates exiftool for media metadata (uses `exiftool-vendored`).
- *Cons*: Still maturing (few users, no releases). Performance may lag on media (OCR/LLM calls). Limited documentation beyond GitHub README. Doesn’t handle Office AST; treats docs at a higher level.
- *Use Cases*: When you **need Microsoft-MarkItDown parity** in Node. E.g. ingest heterogeneous content (office docs, images, media) and convert to Markdown for an AI pipeline. Good if you need OCR and even AI-generated summaries. Not ideal if you only need raw text or parsing control (officeParser would be better then).
- **textract (Node Extract-Text) (Open Source)** – [GitHub](#) by devmehq (MIT). A classic Node text extractor (fork of veljkomatic/textract) that runs external parsers.
- *Formats*: Many common types: HTML, PDF, DOC, DOCX, XLS, XLSX, CSV, PPTX, PNG, JPG, GIF, RTF, etc. [\[78†L621-L629\]](#) . (It spawns command-line tools or libraries under the hood: e.g. `pdftotext` , `antiword` , `unrtf` , etc. See GitHub for full list.) Does not natively support EPUB, ZIP, or complex XML.

- **Outputs:** Plain text string. (It strips formatting and returns text content.) It **does not** output Markdown or rich structure.
- **OCR:** Yes – it can OCR images via Tesseract (if tesseract is installed on the system) [74†L325-L333] (the Node package has hooks for Tesseract). Setup requires installing Tesseract and other backends (antiword, pdftminer, etc.) on the host.
- **Integration:** Node API with callbacks or Promises. Example:

```
textract.fromFileWithPath("file.pdf", (err, text) => {...})
```

Also a CLI (`textract file.docx`). Simple to use, but heavy on native dependencies. Documentation lists each parser.
- **Performance:** Depends on external tools – can be slow (spawning processes) especially for many files. Works reasonably for small/medium tasks. Not clustered.
- **Community & Maintenance:** Mature but quiet. Several forks; original version had many GitHub issues. devmehq's fork (v2.0.0) updated in 2023. ~200★ historically.
- **License:** MIT (open source) [78†L628-L630] .
- **Pros:** Straightforward, covers most office/pdf/image basics [78†L621-L629] . Single API for many formats (so one-stop for text extraction). No LLM or AST complexity.
- **Cons:** Requires many external dependencies (so installation headache). Only plain text output (no Markdown/HTML). Lacks advanced format handling (no table parsing, images detached, etc.). Less actively maintained than newer libs.
- **Use Cases:** Quick-and-dirty text extraction from files in Node when you can install the needed tools. Good for scripts that index documents or feed text to search/AI. Not ideal for heavily structured output.
- **Apache Tika Wrappers (Open Source)** – Node bridges to Apache Tika, the Java “swiss army knife” of parsing. Two main wrappers: **node-tika** (ICIJ, MIT) and **@shelf/tika-text-extract** (shelfio, open/archived). Both require Java/Tika JAR.
- **Formats:** Essentially *everything* that Apache Tika supports – literally *hundreds* of formats (documents, archives, media) including Office, PDF, HTML, images, EPUB, XML, etc. Tika “detects and extracts metadata and text from over a thousand different file types” [51†L275-L282] . (Office, PDF, images via OCR, OpenOffice, archives, many text/media types.)
- **Outputs:** Plain text (or XHTML/metadata if requested). Node-tika lets you get text (`tika.text()`), XHTML (`tika.xhtml()`), metadata, MIME type, language, etc. [79†L391-L400] [79†L405-L413] . shelf's tika-text-extract returns text from a Tika server. No built-in Markdown conversion.
- **OCR:** Yes – Tika can call Tesseract on images (Node-tika has `ocrLanguage` option [79†L329-L337]). Must have Tesseract installed.
- **Integration:** **node-tika** (ICIJ) uses a Java bridge (node-java), requiring JDK 7+ and Python 2 setup [79†L297-L304] . Example: `tika.text('file.pdf', options, callback)` . **tika-text-extract** (shelfio) uses a Tika HTTP server – you must start a Java Tika server JAR (e.g. via `await tte.startServer('tika-server.jar')`), then call `tte.extract(buffer)` [51†L308-L317] . Both are more involved (need Java), but they work in Node contexts.
- **Performance:** Startup overhead (Java, server), but once running Tika is fast. node-tika spawns Java for each call; tika-text-extract keeps a persistent server (better for many calls). Scalability depends on resource needs.

- *Community & Maintenance*: Node-tika (ICIJ) is older (Tika 1.13, MIT); last significant update ~2017. shelf's tika-text-extract (Apache Tika 2.x) has ~16★, archived – not actively maintained [51†L269-L277] . Both are niche.
- *License*: Apache 2 (Tika) for Tika itself; node-tika is MIT [79†L284-L293] .
- *Pros*: **The broadest format coverage possible** (thanks to Tika) – including obscure types. Handles nested archives, rich metadata, language detection, etc. Good OCR support.
- *Cons*: Heavy setup (Java, possible Docker/Lambda tweaks). Output is plain text/XHTML – no formatting beyond that. Not as easy to customize as pure-Node libs. Slightly outdated Node implementations.
- *Use Cases*: If you absolutely need maximum format coverage in a Node service. E.g. enterprise search/indexing where any file type might appear. Or in a Lambda where Tika-server can be baked in. (Otherwise, simpler tools usually suffice.)
- **LibreOffice Conversion (Open Source)** – e.g. [libreoffice-convert](#) (MIT) or calling `soffice` CLI. These use LibreOffice headlessly to convert Office docs to PDF/HTML.
 - *Formats*: All Office formats that LibreOffice supports (DOC/DOCX, PPT/PPTX, XLS/XLSX, ODF, etc.). Also can open some PDFs.
 - *Outputs*: Generally PDF or HTML (and then you must extract text/Markdown from those). Some wrappers offer direct DOCX→PDF or DOCX→TXT.
 - *OCR*: No. Images inside docs may be rasterized in PDF/HTML output but no OCR on images.
 - *Integration*: Requires LibreOffice installed on the machine. Node wrapper `libreoffice-convert` just spawns `soffice` . As easy as `convert.convert(file, '.pdf', ...)` .
 - *Performance*: Resource-heavy and slow (spinning up LibreOffice for each document). Not scalable for large batches.
 - *Community*: Popular in some contexts (LibreOffice is active). Node wrappers are simple.
 - *License*: LGPL or MIT depending on wrapper; LibreOffice itself is MPL/LGPL.
 - *Pros*: Very good fidelity (uses LO's rendering). Handles Office nuances (fonts, layouts). No external paid service.
 - *Cons*: Heavy dependency; cannot run without LO. Limited to LO-supported formats (no EPUB, ZIP, audio, etc.). Output is not Markdown (you'd need another step).
 - *Use Cases*: When you need **pixel-perfect rendering** or PDF output of office files. For example, batch converting company reports. Not ideal if you need structured text output.
- **Nutrient Node.js SDK (Commercial)** – Node SDK for document conversion (via [Nutrient API](#)). Handles DOCX→PDF and other Office conversions in a fully-managed way [84†L13-L22] [84†L27-L35] .
 - *Formats*: Primarily Microsoft Office and PDF. (The blog says DOCX→PDF; presumably also PPTX/XLSX conversions). Not generic text extraction.
 - *Outputs*: Usually PDF or Office formats; text/markdown not its focus.
 - *OCR*: Not specifically (just office rendering).
 - *Integration*: npm package – easy install. Requires account/credentials. Example (from docs): `await client.convert({ file: buffer, from: 'docx', to: 'pdf' })` .
 - *Performance*: Very fast and reliable (cloud-managed). Zero local dependencies.

- **Community:** Commercial product, likely well-supported. No usage numbers publicly.
- **License:** Commercial (proprietary).
- **Pros:** Enterprise-grade service: high reliability, consistent output, no local setup [【84†L25-L33】](#) . Works seamlessly in Node.
- **Cons:** Paid service (cost per doc). Only covers conversions (Office→PDF/etc.), not wide format extraction (no Markdown output). Requires internet connectivity.
- **Use Cases:** When you need guaranteed high-quality conversion (especially DOCX→PDF/XLSX) in production Node apps, and can pay for it. If you don't want to manage LibreOffice or risk compliance issues, this is an option.
- **UniParser (Open Source)** – [GitHub](#) (MIT). A “universal” parser for text from PDF, DOCX, TXT, HTML, Markdown [【67†L261-L270】](#) .
 - **Formats:** PDF, DOCX, plain text, HTML, Markdown [【67†L261-L270】](#) . (No Excel, PPTX, images, or others.)
 - **Outputs:** Plain text. No rich or Markdown output aside from stripping syntax.
 - **OCR:** No OCR support.
 - **Integration:** Minimal; provides functions `parsePDF` , `parseDOCX` , etc. Example: `const { parsePDF, parseDOCX } = require('uniparser'); const text = await parsePDF('file.pdf');`
 - **Performance:** Reasonable for moderate documents. Uses [pdf-parse](#) for PDF and [docx-parser](#) under the hood. Synchronous parse for tiny files.
 - **Community:** Very small (3★ on GitHub). Recent (first released 2024).
 - **License:** MIT.
 - **Pros:** Simple, lightweight, focuses on core formats. Auto-detect mode for file type.
 - **Cons:** Limited format coverage (no PPT, XLS, EPUB, images, etc.). No OCR. Text only (no Markdown formatting). Not mature.
 - **Use Cases:** Quick text extraction in Node from PDF/Word/HTML/MD if you don't need anything fancy. Good for small scripts or prototypes.

Python Libraries

- **MarkItDown (Open Source)** – [GitHub](#) (MIT). Microsoft's original Python tool for exactly this purpose. (Listed here as a reference.)
- **Formats:** Very broad: PDF, PowerPoint, Word, Excel, images (with OCR), audio (speech-to-text), HTML, CSV/JSON/XML, ZIP archives (iterating contents), YouTube, EPUB, etc. [【80†L309-L318】](#) . Essentially all the above and more.
- **Outputs:** Markdown with preserved structure (headings, lists, tables, links) [【80†L301-L309】](#) . Also extracts EXIF metadata for media.
- **OCR:** Yes – uses Tesseract via `pytesseract` for images and transcribes audio via speech APIs.
- **Integration:** Python package (`pip install markitdown[all]`). CLI tool (`markitdown file.docx > out.md`). Also Python API (`from markitdown import convert_file`).
- **Performance:** Solid, but can be slow with media (OCR/transcription) and large files.
- **Community:** High (164k★ on GitHub). Actively maintained by Microsoft and community. Latest release v0.2.0 (Jun 2024).
- **License:** MIT.

- **Pros: Ultimate feature set** (the gold standard). Handles nearly any common format (including media) and outputs Markdown structured for AI. Tuned for ingestion into LLM pipelines.
- **Cons:** Python-only; not Node. Not trivial to embed in a Node app. Requires Python and many optional dependencies (ffmpeg, exiftool, ghostscript, etc.).
- **Use Cases:** Reference for what features we want. If only Python is an option, this is likely the best multi-format converter.
- **textract (Python) (Open Source)** – [textract PyPI](#) by Dean Malmgren (MIT). A general-purpose text extraction library [83†L43-L51] [83†L72-L81] .

- **Formats:** Over 25 file types: `.doc`, `.docx`, `.pptx`, `.xls`, `.xlsx`, PDF, HTML, Markdown, TXT, OpenOffice, email (.eml/.msg), image formats (via OCR), audio (via pocketsphinx), etc. [83†L43-L52] [83†L74-L83] . (Covers CSV/TSV, EPUB, JSON, MP3, etc.)
- **Outputs:** Plain text. (Strips all formatting.)
- **OCR:** Yes – uses Tesseract for images (`gif`, `png`, `tiff`, `jpg`) [83†L54-L63] [83†L77-L85] . Also uses Pocketsphinx for audio transcription (`.wav`, `.mp3`, `.ogg`) [83†L64-L73] [83†L88-L92] .
- **Integration:** Python import `textract`, e.g. `text = textract.process("file.pdf")` . Also CLI (`textract file.ext`). Simple to use once dependencies (antiword, unrtrf, etc.) are installed.
- **Performance:** Depends on chosen parser; in tests it's moderately fast. For large files or many images/audio, performance drops.
- **Community:** Popular (downloads in tens of thousands). Last release 2.0.0 (Apr 2026) [61†L129-L137] . Maintained by Kyle King/Dean Malmgren.
- **License:** MIT.
- **Pros:** Simple single API for many formats [83†L43-L52] [83†L74-L83] . Actively maintained. Good baseline if you don't need Markdown specifically.
- **Cons:** Only plaintext (no Markdown formatting). Sometimes results can be messy (parsers vary in quality). Hard-codes many dependencies (needs antiword, pdftminer, etc.).
- **Use Cases:** Python apps needing quick text extraction. Good for NLP preprocessing (like corpus building) when precise structure isn't needed.

- **Apache Tika (Python) (Open Source)** – [tika-python](#) (Apache 2.0). Python wrapper around Apache Tika (Java).

- **Formats:** Essentially everything Tika supports (1000+ types). (Same as Node Tika: office, PDFs, images, archives, etc.)
- **Outputs:** Plain text, XHTML, metadata. No Markdown.
- **OCR:** Yes (via Tika's OCR integration).
- **Integration:** Runs a local Java Tika server (downloads JAR). Example: `from tika import parser; text = parser.from_file('file.pdf')['content']` .
- **Performance:** JVM startup overhead, but fast thereafter. Good for batch jobs.
- **Community:** Widely used in Python ecosystem. If you value coverage, this is top choice.
- **License:** Apache 2.0 (for Tika).
- **Pros:** Broad coverage, well-tested. Handles complex docs and metadata.
- **Cons:** Requires Java runtime. Python calls Tika via REST (slightly indirect). Again, just raw text output.

- *Use Cases*: When Python needs the broadest possible parser set (e.g. ingesting heterogeneous document stores).
- **Pandoc (Open Source)** – Command-line tool (and [pypandoc](#) binding) for converting between document formats.
- *Formats*: Virtually any markup/document format (Docx, HTML, Markdown, LaTeX, PDF via LaTeX, etc.). Lacks OCR.
- *Outputs*: Many formats (including Markdown, HTML, DOCX, PDF). Very flexible but primarily markup/structured text.
- *OCR*: No.
- *Integration*: CLI or via bindings. Installation can be heavyweight (Haskell binary or via apt).
- *Pros*: Rock-solid for text format conversion. Highly reliable document converter.
- *Cons*: Doesn't parse PDFs well, nor images. No concept of extracting embedded text from arbitrary binaries. Not directly an extraction library.
- *Use Cases*: When input is mostly markup or Office documents, and you need to convert them (e.g. Docx→Markdown). Often used under the hood by others (e.g. MarkItDown uses pandoc for doc→md).

Java Libraries

- **Apache Tika (Open Source)** – The definitive Java library for content extraction (Apache 2.0). Detects and extracts text/metadata from *almost any* file [【51†L275-L282】](#) .
- *Formats*: 1000+ formats (Office, PDF, images, EPUB, XML, ZIP, etc.) [【51†L275-L282】](#) . It bundles PDFBox, POI, etc.
- *Outputs*: Text or XHTML (and metadata). No native Markdown output.
- *OCR*: Yes (if Tesseract is on classpath).
- *Integration*: Java API or CLI. Simply `Tika tika = new Tika(); String text = tika.parseToString(file);` . (Or use Tika server).
- *Performance*: Highly optimized for batch processing. Can process large corpora. Multithreading possible.
- *Community*: Very high (core Apache project). Actively updated (Tika 2.x).
- *License*: Apache 2.0.
- *Use Cases*: Enterprise-grade document pipelines, search indexing, data analysis. Great fallback for any unknown format.
- **Apache POI / PDFBox (Open Source)** – The standard Java libraries for Microsoft Office formats (POI) and PDF (PDFBox) extraction (Apache 2.0).
- *Formats*: POI covers `.xls` / `.xlsx` , `.doc` / `.docx` , `.ppt` / `.pptx` . PDFBox covers PDFs. (No other formats out-of-the-box.)
- *Outputs*: Text, plus you can programmatically access structure (tables, cells, etc.) if needed.
- *OCR*: No (must use something like Tesseract or PDFBox *with* OCR addon).
- *Integration*: Use Java API. Example: `XWPFDocument doc = new XWPFDocument(new FileInputStream("file.docx"));` etc. Or use `PDFTextStripper` for PDFBox.
- *Community*: Very active (hundreds of contributors). Goes hand-in-hand with Tika.

- *License*: Apache 2.0.
- *Use Cases*: Projects focused on Office/PDF where you want programmatic access to document components (e.g. extract table values from Excel, or text from Word). Good for Java-only stacks.
- **iText (Commercial/AGPL)** – Commercial (or AGPL) PDF library. Very powerful PDF manipulation (extraction, creation, editing).
 - *Formats*: PDFs only.
 - *Outputs*: Text, plus can recreate PDF content.
 - *OCR*: No built-in (PDF is text-based).
 - *Pros*: High performance and accuracy for PDF. Good documentation and support (if paid).
 - *Cons*: AGPL license (requires open sourcing or purchase). Not for Office docs.
 - *Use Cases*: Enterprise PDF processing in Java, where a commercial solution is acceptable.
- **Aspose.Total (Commercial)** – Suite of commercial Java libraries (Aspose.Words, Aspose.Cells, Aspose.Slides, Aspose.PDF, etc.).
 - *Formats*: Word, Excel, PowerPoint, PDF, images, email, and more (each component covers a family).
 - *Outputs*: Can convert between formats (e.g. DOCX→TXT/HTML/Markdown/EPUB etc.), or extract text. For example, Aspose.Words can save as Markdown.
 - *OCR*: Aspose.OCR exists (Windows-based), or use Aspose.PDF for some text extraction. Not as integrated as Tesseract.
 - *Integration*: Java (and .NET) libraries. Very easy to use (no external deps).
 - *License*: Commercial (permanent license or subscription).
 - *Pros*: Very broad format support with high fidelity. Actively maintained with support. No external installation needed.
 - *Cons*: Costly. Not open source. (Often overkill for basic needs.)
 - *Use Cases*: Enterprises requiring guaranteed conversions or text extraction across many MS formats, with SLAs. E.g. automated report generation, large-scale document migration.

Comparison Table

Library	Input Formats (Examples)	OCR Support	Outputs (Formats)	License	Type
officeParser	DOCX, PPTX, XLSX, ODT, PDF, RTF, CSV, HTML, Markdown, etc 【72†L330-L339】	Yes (Tesseract.js)	Markdown, HTML, CSV, RTF, PDF, Text, RAG chunks 【72†L330-L339】	MIT (Open)	Node.js Lib
markitdown- js	DOCX, PPTX, XLSX, PDF, HTML, images, media, RSS, ZIP, etc 【74†L278-L287】	Yes (Tesseract.js)	Markdown (+ metadata) 【74†L312-L320】	MIT (Open)	Node.js Lib

Library	Input Formats (Examples)	OCR Support	Outputs (Formats)	License	Type
textract (Node)	HTML, PDF, DOC, DOCX, XLS, XLSX, CSV, PPTX, PNG, JPG, GIF, RTF, etc. 【78†L621-L629】	Yes (Tesseract)	Plain text	MIT (Open)	Node.js Lib
node-tika	1000+ types (Office, PDF, images, archives, etc.)	Yes (Tesseract)	Plain text / XHTML	MIT (Open)	Node.js Lib
tika-text-extract	1000+ types via Tika HTTP server	Yes (Tesseract)	Plain text	Apache 2.0 (Open)	Node.js Lib
LibreOffice Convert	DOCX, PPTX, XLSX, etc. (via LO headless)	No	PDF, HTML	LGPL/MIT (Open)	Node.js Lib
Nutrient SDK	DOCX, PPTX, XLSX (commercial service)	No	PDF, Office	Commercial	Node.js Lib
UniParser	PDF, DOCX, TXT, HTML, MD 【67†L261-L270】	No	Plain text	MIT (Open)	Node.js Lib
MarkItDown (Python)	PDF, PPTX, DOCX, XLSX, images, audio, HTML, CSV/JSON/XML, ZIP, YouTube, EPUB, etc. 【80†L309-L318】	Yes (OCR, speech)	Markdown	MIT (Open)	Python Tool
textract (Python)	PDF, DOC, DOCX, PPTX, XLSX, HTML, MD, CSV/TSV, EPUB, images (OCR), audio (transcription), etc. 【83†L43-L52】 【83†L74-L83】	Yes (OCR, speech)	Plain text	MIT (Open)	Python Lib
Apache Tika (Java/Python)	1000+ types (Office, PDF, images, archives, etc.) 【51†L275-L282】	Yes (OCR)	Plain text / XHTML	Apache 2.0 (Open)	Java/ Python
Apache POI/ PDFBox	DOCX, XLSX, PPTX, PDF	No	Plain text / Object (tables)	Apache 2.0 (Open)	Java Lib
Aspose.Total (Java)	DOCX, XLSX, PPTX, PDF, HTML, images, email, etc.	Limited	Many (DOCX, PDF, HTML, Markdown, etc.)	Commercial	Java Lib

Note: “Output” lists typical uses; nearly all above can be made to produce text/plain or Markdown via additional steps or options.

Top Recommendations for Node.js

For a **production-grade Node.js** solution that approaches MarkItDown’s versatility, the top picks are:

1. **officeParser (Open Source)** – Best all-rounder. It natively handles Word/Excel/PPT plus common formats, yields Markdown/HTML/text, and has built-in OCR [72†L330-L339] . Its output is high-fidelity and structured. This should be the first choice for heavy-duty document processing.
2. **markdown-js (Open Source)** – Closest feature match to Microsoft’s MarkItDown in Node. It covers an extremely broad range of formats [74†L278-L287] and outputs Markdown with metadata. Use this if you need to cover unusual types (YouTube links, RSS, media, etc.) that most extractors skip.
3. **textextract (Node – Open Source)** – Practical fallback. It supports most standard file types (PDF, Office, images) [78†L621-L629] and is easy to use for plain text extraction. It’s less feature-rich (no AST or Markdown), but very reliable for “get me the words” tasks once dependencies are set up.

If a single library doesn’t do it all, a **combined approach** is reasonable. For example:

- Use **officeParser** for Word/Excel/PDF/HTML (with OCR) to get structured output.
- Use **markdown-js** for image/audio-heavy or exotic content.
- Use **textextract** for quick text from miscellaneous formats.
- Optionally supplement with **tika-text-extract** (or a self-hosted Tika server) for any edge-case file types (EPUB, odd archives, etc.) that slip through.

Why MarkItDown Is Hard to Replace

Microsoft’s MarkItDown is unusually comprehensive [80†L309-L318] . It isn’t just a document parser; it also handles **multimedia and AI features** (audio transcription, image captioning via LLM, YouTube summaries, etc.) that most libraries ignore. No single JavaScript library duplicates all of that. In particular, things like:

- **Media Transcription:** MarkItDown can auto-caption images and transcribe audio by calling language models and APIs. Node solutions would need custom code (e.g. AWS Transcribe, Google Vision, or OpenAI) to match this.
- **EEPB/ZIP/Archives:** It iterates inside ZIP/EPUB and processes contents (e.g. converting embedded HTML or docx).
- **High-Fidelity Markdown:** It tries to preserve structure (headings, lists, tables) in output. Many extractors only yield raw text, losing that structure.

Because of this breadth, replacing MarkItDown in Node requires *multiple tools* and possibly custom integration (or calling a Python service).

Sources

- Library documentation and repositories 【72†L330-L339】 【74†L278-L287】 【78†L621-L629】 【79†L291-L299】 【80†L309-L318】 【83†L43-L52】 【83†L74-L83】 【51†L275-L282】 (format/feature support).
 - Library READMEs for output formats and OCR notes 【72†L330-L339】 【74†L325-L333】 【79†L329-L337】 .
 - Community info (GitHub stars, licenses) from project pages 【72†L330-L339】 【78†L628-L630】 【74†L428-L431】 .
-